

UNITÉ 9 · SEMAINE 15

OS natif pour l'IA : agents et LLM dans le système d'exploitation

📅 Semaine 15 · dates à confirmer </> Laboratoire TD8 (corrigé + défi IA) ~ 4 animations

LABORATOIRE TD8 · CORRIGÉ (NOUVEAU)

Un SE agentique avec votre propre micro-LLM

Windows et les distributions Linux commencent à héberger des agents d'IA dans le système. Ce laboratoire construit cette architecture en miniature avec tout le cours: un **micro-LLM** entraîné pour le cours transforme des requêtes en espagnol ou en anglais (il ne comprend que ces deux langues) en un appel d'outil; un **démon** (`agentd`) le reçoit par un socket UNIX, le valide avec une **politique**, demande le **consentement** s'il est destructeur et l'exécute dans un processus enfermé par **Landlock**, en notant tout dans un **journal d'audit**.

</> Compiler et tester

Bash

```
make -C tds                                # compile aussi td8 : tinylm, agentd, agent,
agent_exec
tds/tests/run.sh td8                        # 24 vérifications : modèle, requêtes,
politique, consentement, Landlock, audit
cd tds/bin/td8 && ./tinylm -v "list the files in docs" # requêtes en
espagnol ou en anglais seulement
```

Il faut Linux ≥ 5.13 avec Landlock activé (`cat /sys/kernel/security/lsm`).

L'entraînement est facultatif (les poids sont fournis): `make -C tds/td8 train` (NumPy, ≈ 1 h sur un CPU).

✅ Rapport du TP

Rendez ce TP avec un court rapport en LaTeX, avec le modèle du cours: chaque exercice expliqué, le code qui compte et ce que vous avez exécuté dans le terminal. Vous pouvez utiliser l'IA générative pour écrire le LaTeX: déclarez-le et vérifiez chaque résultat. Modèle, exemple et guide de Prism →

EXERCICES

- | | | | |
|---|--|---|---|
| 1 | Le micro-LLM: données et entraînement | 3 | <code>agentd</code> : socket, pool de threads et identité |
| 2 | L'inférence en C avec <code>mmap</code> et un cache KV | 4 | La politique et le consentement |
| | | 5 | Défense en profondeur: Landlock |
| | | 6 | Audit et réflexion |

TD8 · 1

● Corrigé · testé

Le micro-LLM: données et entraînement

Énoncé. Lisez `train/dataset.py` et `train/train.py`. Qu'apprend exactement le modèle? Pourquoi les noms de fichiers sont-ils générés au hasard? Comment sait-on que le gradient écrit à la main est correct?

Solution

Python `tds/td8/train/dataset.py`

```
"""Synthetic data for the TD8 micro-LLM: requests in Spanish or English →
tool call.
```

```
Every example is a text of UTF-8 bytes:
```

```
Q: lista los archivos de docs\n
A: list_files("docs")\n
```

```
The model only learns to translate the intent into ONE call from a closed
catalogue of tools
(the equivalent of the MCP "connectors" of an agentic OS). File and folder
names are generated
at random so that the model learns to COPY them from the request (attention),
not to memorise them.
None of this is safe by itself: the OS decides whether to run the call
(policy + Landlock in agentd).
"""
```

- Un transformer de style GPT **au niveau des octets** (vocabulaire 256, contexte 128, d = 64, 4 têtes, 3 couches, ≈ 175 mille paramètres), entraîné avec NumPy seul (sans

PyTorch) et une rétropropagation écrite à la main; `python3 train.py --check` compare le gradient analytique avec des différences finies.

- Il apprend à traduire une requête en **un** appel d'un catalogue fermé de 12 outils. Les noms de fichiers aléatoires obligent le modèle à les **recopier** depuis la requête (attention) au lieu de les mémoriser.
- **Résultat réel** (`tds/td8/weights/train_log.json`): 174 656 paramètres, 12 000 étapes (≈ 87 min sur 4 threads CPU), **99,0 %** de correspondance exacte sur 500 nouvelles requêtes; l'outil le plus difficile est `write_note` (89 %), car il faut recopier deux arguments. Poids: 698 688 octets.
- **Leçon des données**: la version précédente (v1) obtenait 99,4 % sur son test, mais au laboratoire elle a changé `viejo.txt` en `viefe.txt` et n'a pas reconnu `atajo` : elle avait vu peu de noms de ce genre. Au lieu de changer le réseau, on a élargi le générateur de données (noms avec tout l'alphabet, sans extension). Le modèle ne vaut que ce que valent ses données, et c'est pourquoi `agentd` ne lui fait jamais confiance.
- Les données contiennent exprès des chemins dangereux (`/etc/passwd`, `../..`): le modèle doit les recopier fidèlement, car décider s'ils sont acceptables **n'est pas son rôle**.

Le voir de l'intérieur dans l'animation A9.2 →

TD8 · 2

● Corrigé · testé

L'inférence en C avec `mmap` et un cache KV

Énoncé. Expliquez comment `tinylm.c` charge les poids sans les copier et pourquoi chaque nouveau token ne coûte qu'un seul passage dans le réseau. Lancez `./tinylm -v` et reliez ses métriques aux unités 4 et 6. Vérifiez que C et NumPy calculent la même chose.

Solution

C `tds/td8/tinylm.c`

```
int tl_load(tl_model *m, const char *path)
{
    memset(m, 0, sizeof *m);
    int fd = open(path, O_RDONLY);
    if (fd == -1)
        return -1;
```

```

struct stat st;
if (fstat(fd, &st) == -1 || st.st_size < HEADER) {
    close(fd);
    errno = EINVAL;
    return -1;
}

/* MAP_PRIVATE + PROT_READ: nobody can modify the weights through this
mapping. */
void *map = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE, fd,
0);
close(fd); /* the mapping keeps the
file alive */
if (map == MAP_FAILED)
    return -1;

const unsigned char *raw = map;
int32_t h[7];
memcpy(h, raw + 8, sizeof h);
if (memcmp(raw, "FSOLLM01", 8) != 0 || h[0] != 1 || h[4] > TL_MAX_LAYERS
|| h[3] % h[5] != 0) {
    munmap(map, (size_t)st.st_size);
    errno = EINVAL;
    return -1;
}
m->vocab = h[1]; m->ctx = h[2]; m->d = h[3]; m->n_layer = h[4]; m->n_head
= h[5]; m->d_ff = h[6];
m->map = map;
m->map_size = (size_t)st.st_size;

/* The pointers point INTO the mapping: no copy (the file is little-
endian float32). */
const float *p = (const float *)(raw + HEADER);
const int d = m->d, f = m->d_ff;
#define TAKE(dst, n) do { (dst) = p; p += (n); } while (0)
TAKE(m->wte, (long)m->vocab * d);
TAKE(m->wpe, (long)m->ctx * d);
for (int l = 0; l < m->n_layer; l++) {
    tl_layer *L = &m->layer[l];
    TAKE(L->ln1_g, d); TAKE(L->ln1_b, d);
    TAKE(L->w_qkv, (long)d * 3 * d); TAKE(L->b_qkv, 3 * d);
    TAKE(L->w_o, (long)d * d); TAKE(L->b_o, d);
    TAKE(L->ln2_g, d); TAKE(L->ln2_b, d);
    TAKE(L->w_fc, (long)d * f); TAKE(L->b_fc, f);
    TAKE(L->w_pr, (long)f * d); TAKE(L->b_pr, d);
}

```

```

    TAKE(m→lnf_g, d); TAKE(m→lnf_b, d);
#undef TAKE
    m→n_params = (long)(p - (const float*)(raw + HEADER));
    if ((size_t)HEADER + (size_t)m→n_params * sizeof(float) ≠ m→map_size)
    {
        munmap(map, m→map_size);
        errno = EINVAL;
        return -1;
    }
    return 0;
}

```

Sortie réelle

```

$ ./tinyllm -v "lista los archivos de docs"
step 0: 'l' 100.0% 'f' 0.0% 'r' 0.0%
step 1: 'i' 100.0% 'q' 0.0% '4' 0.0%
step 2: 's' 100.0% 'g' 0.0% 'q' 0.0%
step 3: 't' 100.0% 'k' 0.0% 'b' 0.0%
step 4: '_' 100.0% '-' 0.0% 'k' 0.0%
step 5: 'f' 100.0% '2' 0.0% 'c' 0.0%
step 6: 'i' 100.0% 'o' 0.0% 'n' 0.0%
step 7: 'l' 100.0% 'L' 0.0% 's' 0.0%
step 8: 'e' 100.0% 'o' 0.0% 'f' 0.0%
step 9: 's' 100.0% '(' 0.0% 'q' 0.0%
step 10: '(' 100.0% ' ' 0.0% 'g' 0.0%
step 11: '"' 100.0% ' ' 0.0% 'R' 0.0%
step 12: 'd' 100.0% 'k' 0.0% 'b' 0.0%
step 13: 'o' 100.0% 'i' 0.0% 'e' 0.0%
step 14: 'c' 100.0% 'k' 0.0% 'f' 0.0%
step 15: 's' 100.0% 'q' 0.0% 'f' 0.0%
step 16: '"' 100.0% 'z' 0.0% 'k' 0.0%
step 17: ')' 100.0% ',' 0.0% ':' 0.0%
step 18: '\n' 100.0% 'A' 0.0% ' ' 0.0%
list_files("docs")

model      : 174656 parameters, 3 layers, 4 heads, d=64, context 128 bytes
weights    : 698688 bytes mapped with mmap (MAP_PRIVATE, PROT_READ)
KV cache   : 201984 bytes per request (+ scratch)
tokens     : 33 of prompt (prefill) + 19 generated (decode)
time       : load 0.042 ms, inference 9.08 ms (5730 tokens/s)
page faults: 2 during load, 47 during the first inference (minor; major: 0)

```

Sortie réelle

```
$ python3 train/compare.py ./tinylm 200
200/200 respuestas idénticas entre NumPy y C
```

`mmap` avec `PROT_READ` et `MAP_PRIVATE` renvoie une adresse; les pointeurs de chaque couche pointent à l'intérieur du fichier projeté. Rien n'est lu avant que le premier token touche les poids: c'est pourquoi les défauts de page apparaissent pendant la première inférence et non pendant le chargement. Le cache KV stocke, par couche et par position, les clés et valeurs déjà calculées: chaque nouvel octet ne calcule que sa propre ligne. L'inférence en C et celle en NumPy donnent **les mêmes réponses** sur 200 nouvelles requêtes.

TD8 · 3

● Corrigé · testé

agentd : socket, pool de threads et identité

Énoncé. Lancez `agentd` avec un espace de travail et envoyez-lui des requêtes avec `agent` (en espagnol ou en anglais). Identifiez dans le code quelle partie vient de chaque unité du cours. Comment `agentd` sait-il qui lui parle?

Solution

C tds/td8/agentd.c

```
static void serve(int client)
{
    struct ucred cred = { 0 };
    socklen_t len = sizeof cred;
    if (getsockopt(client, SOL_SOCKET, SO_PEERCREDS, &cred, &len) == -1)
        return;

    char req[MAX_REQ], out[256], why[PATH_MAX + 64];
    if (read_line(client, req, sizeof req) == 0)
        return;

    /* 1. The model proposes. */
    int steps = tl_generate(&model, req, out, sizeof out, NULL, NULL);
    if (steps < 0) {
        send_str(client, "error: request too long\n");
    }
}
```

```

        audit(cred.uid, cred.pid, req, "-", "deny", "too long");
        return;
    }
    send_fmt(client, "model: %s\n", out);

    /* 2. The operating system disposes. */
    call_t c;
    if (parse_call(out, &c) == -1) {
        send_str(client, "policy: deny (the model produced an invalid
call)\n");
        audit(cred.uid, cred.pid, req, out, "deny", "invalid call");
        return;
    }
    decision_t d = policy(&c, why, sizeof why);
    send_fmt(client, "policy: %s (%s)\n", DECISION[d], why);
    if (d == DENY) {
        audit(cred.uid, cred.pid, req, out, "deny", why);
        return;
    }
    if (d == ASK) {
        char answer[16];
        send_fmt(client, "ask: allow the agent to run %s? [y/N]\n", out);
        read_line(client, answer, sizeof answer);
        if (answer[0] != 'y' && answer[0] != 'Y') {
            send_str(client, "cancelled by the user\n");
            audit(cred.uid, cred.pid, req, out, "ask", "refused by the
user");
            return;
        }
    }

    /* 3. Sandboxed execution. */
    const char *outcome = run_tool(client, &c);
    audit(cred.uid, cred.pid, req, out, d == ASK ? "ask:approved" : "allow",
outcome);
}

```

Sortie réelle

```
$ ./agent "lista los archivos de docs"
model: list_files("docs")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
2 entries in docs
-      0  plan.md
-      0  syllabus.pdf
```

Sortie réelle

```
$ ./agent "muéstrame notas.txt"
model: read_file("notas.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Laboratorio TD8
fork, exec, pipe, mmap, sockets
```

Sortie réelle

```
$ ./agent "busca 'error' en log.txt"
model: search("error", "log.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
log.txt:2: error: disk full
log.txt:4: error: timeout
2 matching line(s) for "error"
```

Sortie réelle

```
$ ./agent "¿qué versión del núcleo tengo?"
model: system_info()
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Linux 7.0.0-34-generic (x86_64), 28 CPUs
memory: 32.8 GB total, 6.9 GB free; 3110 processes; up 4 h 7 min
```

Sortie réelle

```
$ for i in $(seq 8); do ./agent "lista mis archivos" & done; wait
8
```

PIÈCE D'AGENTD	UNITÉ
Socket UNIX avec permissions 0600	U8 · U3
<code>SO_PEERCRED</code> : uid et pid du client, fournis par le noyau	U8
Pool de threads avec une file bornée (sémaphores + mutex)	U7
Poids partagés avec <code>mmap</code>	U6
<code>fork</code> + <code>execv</code> de l'exécuteur; sortie par un tube	U4 · U5
<code>SIGTERM</code> pour arrêter; <code>SIGPIPE</code> ignoré	U6

TD8 · 4

● Corrigé · testé

La politique et le consentement

Énoncé. Demandez à l'agent de supprimer un fichier de l'espace de travail (en répondant oui puis non), de supprimer `/etc/passwd`, de lire `../../etc/shadow` et de lire `atajo`, un lien symbolique vers `/etc/passwd` dans l'espace de travail. Qui décide dans chaque cas: le modèle ou le SE ?

Solution



C tds/td8/agentd.c

```
static int inside_workspace(const char *rel, char *why, size_t whysz)
{
    if (rel[0] == '/' || rel[0] == '~') {
        snprintf(why, whysz, "absolute path outside the workspace");
        return 0;
    }
    if (strcmp(rel, "..") == 0 || strncmp(rel, "../", 3) == 0 || strstr(rel,
"/../") || strstr(rel, "/..") ) {
        snprintf(why, whysz, "'..' leaves the workspace");
        return 0;
    }
    /* Resolve symbolic links: the file (or, if it does not exist yet, its
directory). */
    char full[PATH_MAX * 2], real[PATH_MAX];
    snprintf(full, sizeof full, "%s/%s", workspace, rel);
    if (realpath(full, real) == NULL) {
        char *slash = strrchr(full, '/');
        *slash = '\0';
        if (realpath(full, real) == NULL) {
            snprintf(why, whysz, "directory does not exist");
            return 0;
        }
    }
    size_t n = strlen(workspace);
    if (strncmp(real, workspace, n) != 0 || (real[n] != '\0' && real[n] !=
'/')) {
        snprintf(why, whysz, "resolves to %s, outside the workspace", real);
        return 0;
    }
    return 1;
}

static decision_t policy(const call_t *c, char *why, size_t whysz)
{
    const tool_t *t = c->tool;
    if (strcmp(t->name, "none") == 0) {
        snprintf(why, whysz, "no tool for this request");
        return DENY;
    }
    if (policy_on && t->path_arg >= 0 && !inside_workspace(c->args[t
->path_arg], why, whysz))
        return DENY;
    if (t->consent) {
```

```

    snprintf(why, whysz, "destructive action: needs the user's consent");
    return ASK;
}
    snprintf(why, whysz, policy_on ? "tool in the catalogue, path inside the
workspace" : "POLICY DISABLED");
    return ALLOW;
}

```

Sortie réelle

```

$ ./agent -n "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> n (automatic answer)
cancelled by the user

```

Sortie réelle

```

$ ./agent -y "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> y (automatic answer)
[sandbox: Landlock ABI 8, workspace only]
viejo.txt deleted

```

Sortie réelle

```

$ ./agent "borra /etc/passwd"
model: delete_file("/etc/passwd")
policy: deny (absolute path outside the workspace)

```

Sortie réelle

```

$ ./agent "lee ../../etc/shadow"
model: read_file("../../etc/shadow")
policy: deny ('..' leaves the workspace)

```

Sortie réelle

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: deny (resolves to /etc/passwd, outside the workspace)
```

Sortie réelle

```
$ ./agent "cuéntame un chiste"
model: none()
policy: deny (no tool for this request)
```

Le modèle propose toujours ce qu'on lui a demandé — y compris

`delete_file("/etc/passwd")` — ; c'est `agentd` qui décide: les chemins absolus, `..` et les liens qui s'échappent (résolus avec `realpath`) sont refusés; tout ce qui est destructeur demande l'avis de l'utilisateur.

Essayez avec vos propres requêtes dans l'animation A9.4 (en espagnol ou en anglais) →

TD8 · 5

● Corrigé · testé

Défense en profondeur: Landlock

Énoncé. Relancez `agentd` avec `--no-policy` (pour simuler un bogue de la politique) et répétez les attaques. Puis lancez `agent_exec` directement avec et sans `--no-sandbox`. Quelle couche vous arrête maintenant?

Solution

```
C tds/td8/agent_exec.c

static int sandbox(const char *workspace)
{
    int abi = ll_create(NULL, 0, LANDLOCK_CREATE_RULESET_VERSION);
    if (abi < 1)
        return 0;

    __u64 fs = LANDLOCK_ACCESS_FS_EXECUTE | LANDLOCK_ACCESS_FS_WRITE_FILE |
        LANDLOCK_ACCESS_FS_READ_FILE |
```

```

        LANDLOCK_ACCESS_FS_READ_DIR | LANDLOCK_ACCESS_FS_REMOVE_DIR |
LANDLOCK_ACCESS_FS_REMOVE_FILE |
        LANDLOCK_ACCESS_FS_MAKE_CHAR | LANDLOCK_ACCESS_FS_MAKE_DIR |
LANDLOCK_ACCESS_FS_MAKE_REG |
        LANDLOCK_ACCESS_FS_MAKE SOCK | LANDLOCK_ACCESS_FS_MAKE_FIFO |
LANDLOCK_ACCESS_FS_MAKE_BLOCK |
        LANDLOCK_ACCESS_FS_MAKE_SYM;

    if (abi ≥ 2) fs |= LANDLOCK_ACCESS_FS_REFER;
    if (abi ≥ 3) fs |= LANDLOCK_ACCESS_FS_TRUNCATE;
    if (abi ≥ 5) fs |= LANDLOCK_ACCESS_FS_IOCTL_DEV;

    struct landlock_ruleset_attr attr = { .handled_access_fs = fs };
    size_t size = sizeof(__u64); /* ABI 1-3:
only the fs field */
    if (abi ≥ 4) {
        attr.handled_access_net = LANDLOCK_ACCESS_NET_BIND_TCP |
LANDLOCK_ACCESS_NET_CONNECT_TCP;
        size = offsetof(struct landlock_ruleset_attr, handled_access_net) +
sizeof(__u64);
    }
    if (abi ≥ 6) {
        attr.scoped = LANDLOCK_SCOPE_ABSTRACT_UNIX_SOCKET |
LANDLOCK_SCOPE_SIGNAL;
        size = sizeof attr;
    }
    int rs = ll_create(&attr, size, 0);
    exit_if(rs == -1, "landlock_create_ruleset");

    /* Everything inside the workspace (minus executing programs), read-only
/proc. */
    exit_if(ll_add_path(rs, workspace, fs & ~
(__u64)LANDLOCK_ACCESS_FS_EXECUTE) == -1, "landlock_add_rule workspace");
    exit_if(ll_add_path(rs, "/proc", LANDLOCK_ACCESS_FS_READ_FILE |
LANDLOCK_ACCESS_FS_READ_DIR) == -1,
        "landlock_add_rule /proc");

    exit_if(prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) == -1, "prctl");
    exit_if(syscall(SYS_landlock_restrict_self, rs, 0) == -1,
"landlock_restrict_self");
    close(rs);
    return abi;
}

static void limits(void)
{

```

```
struct rlimit cpu = { 2, 3 }, fsize = { 1 << 20, 1 << 20 }, nofile = {
64, 64 };
exit_if(setrlimit(RLIMIT_CPU, &cpu) == -1, "setrlimit cpu");      /*
SIGXCPU after 2 s */
exit_if(setrlimit(RLIMIT_FSIZE, &fsize) == -1, "setrlimit fsize"); /*
SIGXFSZ past 1 MiB */
exit_if(setrlimit(RLIMIT_NOFILE, &nofile) == -1, "setrlimit nofile");
alarm(5);                                                         /*
SIGALRM: real time */
}
```

Sortie réelle

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: atajo: Permission denied
```

Sortie réelle

```
$ ./agent "lee /etc/passwd"
model: read_file("/etc/passwd")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/passwd: Permission denied
```

Sortie réelle

```
$ ./agent_exec workspace read_file /etc/hostname; ./agent_exec --no-sandbox
workspace read_file /etc/hostname
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/hostname: Permission denied
[sandbox: DISABLED]
host
```

Avant de toucher à quoi que ce soit, `agent_exec` s'enferme: limites de CPU, de taille de fichier et de descripteurs (`setrlimit`), une alarme de 5 s, `no_new_privs` et une règle Landlock qui n'autorise que l'espace de travail (et `/proc` en lecture seule), refuse les connexions TCP et les signaux hors du bac à sable. Même avec la politique désactivée, le **noyau** répond `Permission denied`. Le même programme, sans bac à sable, lit bien le fichier.

TD8 · 6

● Corrigé · testé

Audit et réflexion

Énoncé. Lisez `audit.log`. Quelles informations permettent de reconstituer ce qu'a fait l'agent et au nom de qui ? Que manque-t-il à ce SE agentique pour un usage réel ?

Solution



Sortie réelle

```

2026-09-30T13:14:12 uid=1000 pid=364726 request="lista los archivos de docs"
call=list_files("docs") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364735 request="muéstrame notas.txt"
call=read_file("notas.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364744 request="¿cuántas líneas tiene
main.c?" call=count_lines("main.c") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364753 request="busca 'error' en log.txt"
call=search("error", "log.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364762 request="anota 'examen el lunes' en
agenda.md" call=write_note("agenda.md", "examen el lunes") decision=allow
outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364771 request="crea la carpeta proyectos"
call=make_dir("proyectos") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364780 request="¿cuánto espacio libre
queda?" call=disk_usage() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364789 request="¿qué versión del núcleo
tengo?" call=system_info() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364798 request="show running processes"
call=processes() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364807 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask outcome=refused by the user
2026-09-30T13:14:13 uid=1000 pid=364815 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask:approved outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364824 request="borra /etc/passwd"
call=delete_file("/etc/passwd") decision=deny outcome=absolute path outside
the workspace
2026-09-30T13:14:13 uid=1000 pid=364832 request="lee ../etc/shadow"
call=read_file("../etc/shadow") decision=deny outcome='..' leaves the
workspace
2026-09-30T13:14:13 uid=1000 pid=364840 request="lee atajo"
call=read_file("atajo") decision=deny outcome=resolves to /etc/passwd,
outside the workspace
2026-09-30T13:14:13 uid=1000 pid=364848 request="cuéntame un chiste"
call=none() decision=deny outcome=no tool for this request
2026-09-30T13:14:13 uid=1000 pid=364873 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364875 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364877 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364876 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364880 request="lista mis archivos"

```

```
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364879 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364881 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364878 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
```

- Chaque ligne contient la date, l'uid et le pid du client (fournis par le noyau, pas par le client), la requête, l'appel proposé, la décision et le résultat : elle distingue ce qu'a fait l'agent de ce qu'a demandé l'utilisateur, comme l'exige Windows avec ses comptes d'agent.
- Il lui manque, entre autres : un utilisateur propre à l'agent (aujourd'hui il tourne sous le compte de l'enseignant), un filtre `seccomp`, des limites de mémoire avec les cgroups, la protection du journal contre l'agent lui-même et une défense contre l'injection de prompt dès que l'agent lit le contenu des fichiers.

Liste de vérification

Défi avec l'IA générative · Un nouvel outil pour l'agent... et une attaque

Objectif. Ajoutez un outil `copy_file(source, destination)` au SE agentique : générez des données, réentraînez le micro-LLM, étendez la politique et l'exécuteur. Puis essayez de faire copier par l'agent un fichier hors de l'espace de travail et vérifiez quelle couche l'arrête.

Prompt de départ. Utilisez-le dans l'outil de votre choix (Claude, ChatGPT, Gemini, Copilot, Codex...) avec la version *corrigée* du laboratoire comme contexte. Demandez d'abord un plan et relisez chaque étape avant de l'accepter.

Prompt

Je travaille avec un petit SE agentique en C : un micro-LLM (tinyllm.c) transforme des requêtes en appels d'outils, agentd applique une politique et agent_exec exécute l'outil dans un bac à sable Landlock [coller dataset.py, la table TOOLS d'agentd.c et le main() d'agent_exec.c]. Je veux ajouter copy_file(source, destination). Guide-moi pour : (1) ajouter des modèles en espagnol et en anglais à dataset.py (le micro-LLM ne comprend que ces deux langues) ; (2) réentraîner et mesurer la précision avant et après ; (3) étendre la politique pour que LES DEUX arguments restent dans l'espace de travail ; (4) implémenter la copie dans agent_exec. Ensuite, aide-moi à concevoir 5 requêtes malveillantes (chemins absolus, '..', liens symboliques, injection dans le texte) et à prédire quelle couche arrête chacune : l'analyseur, la politique ou Landlock.

Comment vérifier le résultat

- La précision du modèle réentraîné sur les outils précédents ne baisse pas de plus de 2 points (`train.py --eval`).
- `copy_file` avec une destination hors de l'espace de travail se termine par `policy: deny`, et avec `--no-policy` par `Permission denied` (Landlock).
- Le fichier `audit.log` enregistre les 5 tentatives avec leur décision.

Déclarez votre usage de l'IA : outil, prompts pertinents, ce que vous avez vérifié vous-même et ce que vous avez corrigé. L'important est que vous puissiez expliquer et vérifier chaque décision.